Práctica 4: Concurrencia

Rodrigo García Carmona
rodrigo.garcia@upm.es



Esta práctica es individual y sirve tanto para desarrollar como para poner a prueba los conocimientos adquiridos. Realizar esta práctica es recomendable para aumentar las probabilidades de éxito en los exámenes teóricos.

1. Preparación de la práctica

El objetivo de esta práctica es aprender a utilizar hebras (*threads*) para implementar programas que hagan uso de la concurrencia. Se estudiarán los siguientes conceptos:

- Implementación de hebras.
- Arranque, parada y sincronización de hebras.
- Uso de la concurrencia para implementar simulaciones.
- Uso de la concurrencia para paralelizar trabajos usando tareas.

Trabajo previo al laboratorio

El tiempo de las clases de laboratorio es muy valioso, ya que en ellas tendrá a su disposición a los profesores para resolver cualquier duda que le pueda surgir. Por tanto, para facilitar al máximo el aprovechamiento de este tiempo, las prácticas se han dividido en varias secciones. La primera de ellas ("preparación de la práctica") puede ¡y debe! realizarse **antes de la sesión de laboratorio** propiamente dicha. Leer esta sección antes de entrar a clase hará que su tiempo en el laboratorio sea más productivo y provocará que las dudas más importantes le surjan **durante la clase**, y **no después**, cuando se encuentre estudiando solo en su casa y la única opción que le quede sea escribir un email al profesor.

De hecho, es especialmente importante evitar que se den situaciones como esta la noche antes de un examen.

1.1. Sistemas simples y complejos

Existen, a grandes rasgos, dos tipos de sistemas en lo que a la capacidad para modelarlos y calcular qué resultado van a producir respecta:

- Los **sistemas simples**, caracterizados por un comportamiento predecible, pues está basado en un conjunto de reglas. Su resultado puede calcularse recurriendo a la aplicación de estas reglas sobre las condiciones iniciales en las que se encuentra el sistema.
- Los **sistemas complejos**, compuestos de varios componentes que, al interactuar entre sí, producen comportamientos de un orden superior. La principal característica de los sistemas complejos es que no es posible predecir estos comportamientos a partir de las características individuales de los componentes que los producen.

1.1.1. Sistemas simples

Los sistemas simples son la piedra angular de la física clásica y la ingeniería tradicional, en la que los problemas se descomponen en partes más pequeñas, que son resueltas de forma independiente. Se caracterizan por su predictibilidad y su dependencia de un conjunto finito y conocido de reglas y condiciones iniciales. Poseen las siguientes características:

- **Solución analítica:** El comportamiento de un sistema simple está descrito por completo mediante una fórmula o un conjunto de ecuaciones con una solución analítica. Si se conocen las condiciones iniciales, el estado futuro del sistema puede calcularse sustituyendo las variables de las ecuaciones por estas condiciones iniciales.
- **Linealidad o no linealidad limitada:** En los sistemas simples, un cambio en las condiciones iniciales produce un cambio similar en el resultado.
- **Predictibilidad a largo plazo:** Calcular el estado de un sistema simple a largo plazo no es sustancialmente más difícil que hacerlo a corto plazo. Muchas veces, es algo tan sencillo como cambiar el valor de la variable que representa el tiempo.
- **Reduccionismo:** El reduccionismo es una estrategia que permite calcular el comportamiento de un "todo" a través de la suma de los comportamientos de sus partes. Los sistemas simples son reduccionistas en tanto en cuanto basta con entender qué hacen cada una de sus partes constituyentes para entender el sistema en su conjunto.

Un ejemplo típico de un sistema simple es la trayectoria de un proyectil: una vez se conocen la masa, la velocidad inicial y las fuerzas aplicadas, su posición en cualquier momento del futuro es perfectamente calculable mediante la aplicación de un conjunto de ecuaciones.

En los sistemas simples, **la simulación no es una necesidad**, sino una herramienta de conveniencia. En el ejemplo del proyectil, una simulación podría usarse para visualizar la trayectoria o para calcular soluciones intermedias, pero no es la única vía para obtener el resultado (qué posición ocupa el proyectil al final de su trayectoria y cuándo la alcanza).

1.1.2. Sistemas complejos

Los sistemas complejos están formados por agentes o elementos (los componentes del sistema) que actúan o reaccionan de forma independiente, cada uno obedeciendo sus propias reglas. De las relaciones entre agentes y las interacciones entre ellos surgen una serie de efectos (el resultado del sistema). Los sistemas complejos están cada vez más presentes en la ingeniería y aparecen constantemente en las ciencias, especialmente en las de la vida.

El comportamiento de los sistemas complejos es inherentemente difícil de modelar, debido a las interacciones entre los componentes que los integran. Poseen las siguientes características:

- **Múltiples agentes concurrentes:** El sistema se compone de un gran número de agentes que operan en paralelo, es decir, de forma concurrente. Cada agente obedece sus propias reglas, que pueden ser más o menos sencillas.
- **No linealidad:** Debido a fenómenos como los bucles de realimentación (entre otros), los sistemas complejos tienden a producir resultados muy diferentes frente a pequeños cambios en las condiciones iniciales.
- **Dependencia de la secuencia de interacciones:** El estado futuro de un sistema complejo no puede predecirse únicamente a partir del estado actual, sino que también se necesita conocer los pasos intermedios entre el actual y el que se desea conocer.
- **Comportamiento emergente:** Los sistemas complejos exhiben un "patrón global" que surge espontáneamente de las interacciones locales. Este comportamiento, si bien no está expresado de forma explícita en ninguna parte concreta del sistema es, sin embargo, indudablemente real. Además, este patrón global no puede predecirse mediante una simple agregación del comportamiento de las partes.

Complejo no es lo mismo que complicado

Los sistemas simples pueden ser muy complicados (las fórmulas que los regulan son difíciles de calcular o tienen muchas variables), pero eso no hace que sean complejos; ningún comportamiento emerge de ellos. De la misma forma, los sistemas complejos pueden ser sencillos (cada agente, individualmente, se comportan siguiendo unas reglas que pueden ser elementales), pero eso no hace que sean simples.

Este conjunto de características nos lleva a un conclusión sumamente importante: en cuanto el número de agentes o componentes de un sistema complejo crece a partir de cierto punto, es matemáticamente imposible escribir una fórmula cerrada que describa el estado del sistema a largo plazo.

Además, los sistemas complejos exhiben una dependencia extrema a las condiciones iniciales: un cambio minúsculo en una condición inicial, al propagarse a través de las interacciones concurrentes de los agentes, se amplifica hasta producir resultados completamente diferentes. Este fenómeno es conocido popularmente como el "efecto mariposa".

Existen numerosos ejemplos de sistemas complejos, que van desde lo más pequeño (una célula) hasta lo más grande (el clima o el ecosistema). En todos ellos existe indudablemente un comportamiento global que es, a la vez, irreductible. Si piensa en el desplazamiento de una bandada de pájaros, en la congestión en el transporte público de una ciudad o en el metabolismo humano, será capaz de reconocer patrones globales. En todos estos ejemplos, una predicción a largo plazo es imposible sin una simulación que, en muchos casos, deberá incluso ejecutarse múltiples veces con pequeñas variaciones para entender el abanico de resultados posibles.

Un ejemplo muy sencillo de enteder (y a la vez sorprendentemente divertido) de todo esto es el Juego de la Vida de Conway, en el que un conjunto de reglas sencillas generan patrones que se "autoreplican" y se "desplazan". Un ejemplo de patrón puede verse en la Figura 1.

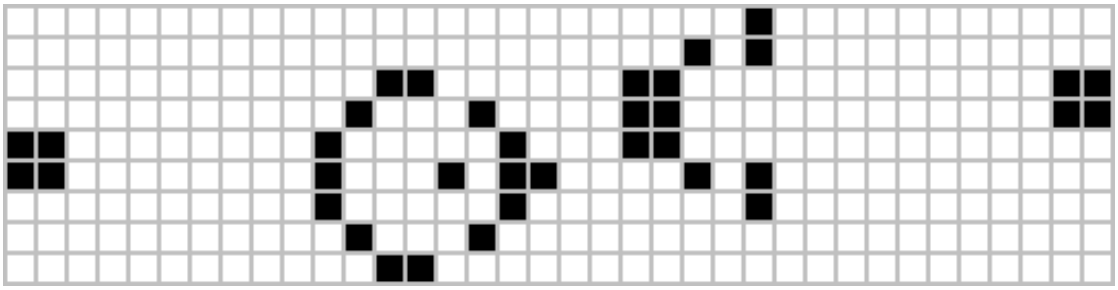


Figura 1: Patrón "Gosper glider gun" en el Juego de la Vida de Conway.

De hecho, para algunos autores, el universo en su conjunto no es sino un gran sistema complejo, del que todos formamos parte.

Vamos a ponernos profundos

La existencia de la complejidad emergente plantea preguntas sumamente profundas sobre dos problemas filosóficos: la consciencia y el libre albedrío.

Y es que quizá la consciencia (la capacidad de experimentar el universo) y el libre albedrío (la capacidad para tomar decisiones intencionalmente) no sean sino comportamientos emergentes que aparecen en uno de los sistemas complejos más grandes que conocemos: el cerebro humano, compuesto por miles de millones de neuronas. Esto es lo que postulan varias teorías de la consciencia, siendo la más popular (aunque no la única) la Teoría de la Información Integrada.

Desde esta perspectiva, el libre albedrío no sería una fuerza que rompe las leyes de la física, sino un fenómeno emergente a un nivel de complejidad tan alto que las interacciones no lineales y la sensibilidad a las condiciones iniciales hacen que el comportamiento del sistema sea impredecible. Y aquí se está usando la palabra "impredecible" con el significado completo y literal de la misma. Es decir no es que el conjunto de decisiones que toma un humano sea **impredecible en la práctica** debido a que hay variables ocultas o mecanismos desconocidos, sino que es **realmente impredecible**: solo una "ejecución" completa del sistema complejo (es decir, la vida de la propia persona) podría llegar al mismo resultado. Esta teoría podría interpretarse, desde un punto de vista religioso, como que ni siquiera un Dios omnisciente sería capaz de conocer el destino de una persona sin antes "instanciarla".

No obstante, para que esta interpretación sea factible, la propia causalidad debe de ser en sí misma uno de los fenómenos emergentes de un sistema complejo, algo que el neurocientífico Erik Hoel parece haber demostrado en un artículo publicado en 2025. Si desea profundizar más en esto, le recomiendo la lectura de este resumen del artículo, muy fácil de entender.

1.2. Simulación de sistemas complejos

De lo explicado hasta ahora se puede concluir que la única forma de predecir el resultado de un sistema complejo es ejecutar una simulación lo más fidedigna posible del mismo. Esta simulación deberá instanciar a tantos agentes como posea el sistema complejo, dejando que estos se comporten de forma independiente, obedeciendo cada uno las reglas que lo gobiernan.

Y esto es, precisamente, lo que nos permite hacer la programación concurrente. En ella, cada una de las hebras representará a un agente, que se ejecutará por su cuenta e interactuará con el resto del sistema. Si el sistema ha sido modelado correctamente, esta técnica suele arrojar predicciones muy fidedignas, convirtiéndose en una herramienta de predicción y análisis excelente.

1.3. Modelado del servicio de Urgencias de un hospital

El servicio de Urgencias de un hospital es un sistema complejo paradigmático, pues posee:

- **Agentes independientes:** Médicos, enfermeros, pacientes, equipos de observación, personal de triaje...
- **Interacciones no lineales:** La llegada de un paciente crítico (una perturbación pequeña) consume múltiples recursos simultáneamente (un médico, un equipo, una sala) durante un tiempo impredecible.
- **Emergencia:** El tiempo de espera promedio, la ocupación de camas y la saturación del personal son propiedades emergentes que no se pueden calcular directamente, sino que surgen de la simulación de las interacciones concurrentes minuto a minuto.

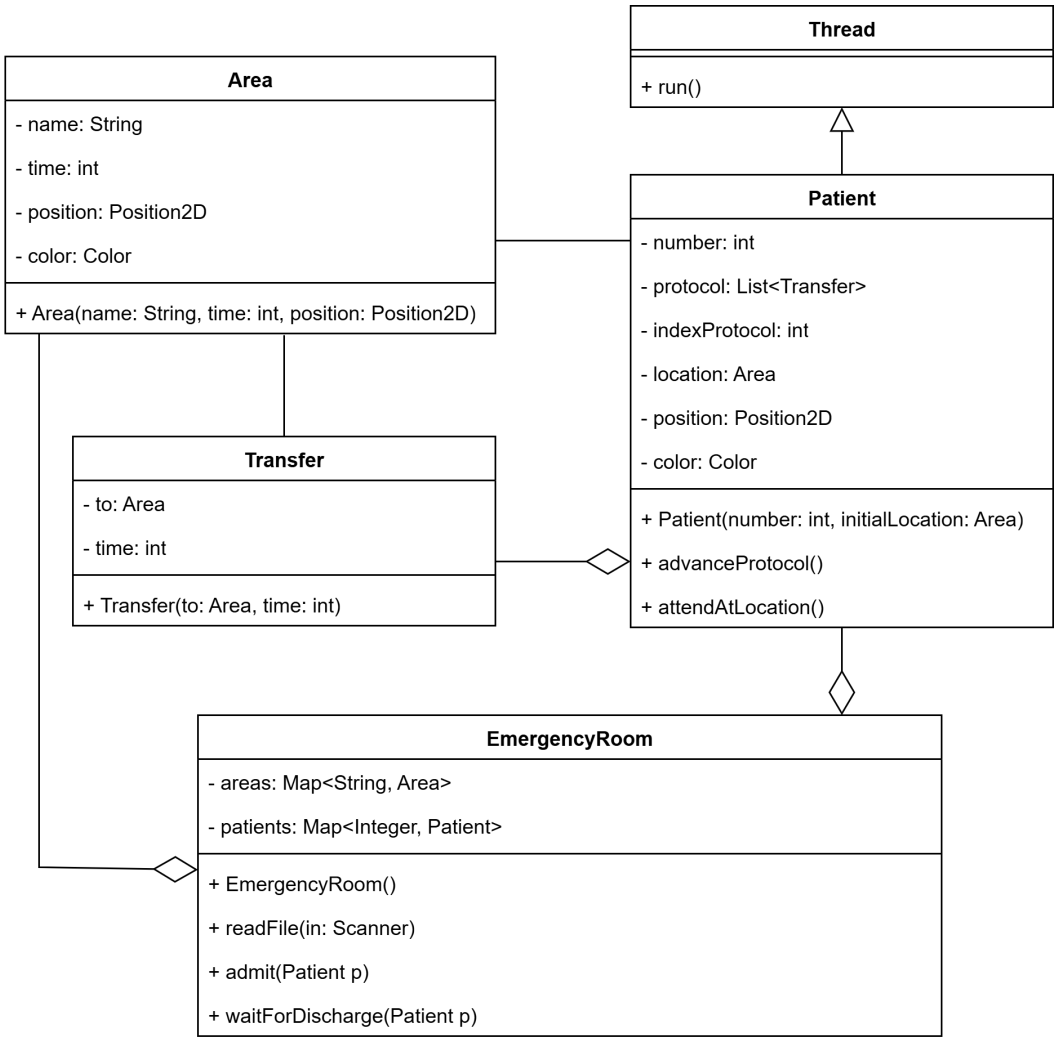


Figura 2: Diagrama de clases del simulador del servicio de Urgencias.

En la **primera parte** (¡ojo, hay dos!) de esta práctica se va a modelar un servicio de Urgencias simplificado, con las clases que aparecen en la Figura 2. En el diagrama se han omitido, por claridad, los métodos `getX()`, `setX()`, `equals()` y `toString()` de cada clase.

Vamos a analizar una por una las clases del diagrama, para explicar su propósito:

- `Area` representa cada una de las secciones del servicio de Urgencias del hospital. Tiene dos atributos que afectan a la simulación: `name`, el nombre de la sección; y `time`, que es el tiempo que tardará en ser atendido (en milisegundos) un paciente. Los otros dos atributos restantes (`position` y `color`) solo afectan a la forma en la que la sección será representada por la interfaz gráfica, y no deberá preocuparse de ellos.

- `Transfer` modela el desplazamiento de un paciente hasta una sección distinta a aquella en la que se encuentra. Posee dos atributos: `to`, que indica el `Area` hacia la que se mueve el paciente; y `time`, que es el tiempo (en milisegundos) que llevará este movimiento.

- `Patient` es la clase que encapsula el comportamiento de un paciente individual. Cada paciente está identificado por un `number`, que será un código único, para preservar su anonimato. Además también posee el atributo `location`, que representa el `Area` en la que se encuentra el paciente en el momento actual y que deberá estar siempre actualizado.

  Todo paciente tiene también un `protocol`, que no es más que una `List` ordenada de `Transfers` que modela el recorrido que seguirá dicho paciente por el servicio de Urgencias según vaya avanzando el tiempo. Para saber en qué punto del protocolo se encuentra el paciente en un momento dado se utiliza el atributo `indexProtocol`, que empieza valiendo 0 cuando el `Patient` es creado.

  Como en el caso de `Area`, `Patient` también cuenta con los atributos `position` y `color`, que no afectan a la simulación; solo son utilizados por la interfaz gráfica.

  Fíjese además en que la clase `Patient` también tiene dos métodos:

  - `advanceProtocol()`, que hace que el paciente avance al siguiente paso en el protocolo.
  - `attendAtLocation()`, que hace que el paciente espere en el `Area` en la que se encuentra el tiempo que esta última determine.

- Por último, la clase `EmergencyRoom` representa el servicio de Urgencias en su conjunto. Como tal, cuenta con un registro de `areas` y `patients` (`Maps` en los que las claves son el nombre de cada `Area` y el número de cada `Patient`, según corresponda), así como los siguientes métodos:

  - `readFile(Scanner in)`, que se usa para cargar la información del servicio (pacientes incluídos) a partir de un fichero.
  - `admit(Patient p)`, que sirve para empezar a atender al `Patient` que se pasa como argumento.
  - `waitForDischarge(Patient p)`, que espera hasta que el `Patient` recibido termine su protocolo.

Estas clases se encuentran en el paquete `es.upm.dit.aled.lab4.er`. Dedique un tiempo a leerlas y entender como funcionan.

Responda a las siguientes preguntas:

- En el diagrama, ¿qué significa la flecha que va desde `Patient` hasta `Thread`?

- ¿Qué metodo de la clase `Patient` no aparece dentro de dicha clase en el diagrama (aparte de `getX()`, `setX()`, `equals()` y `toString()`)? ¿Para qué sirve dicho método? ¿A qué clase pertenece?

- En esta simulación simplificada de un servicio de Urgencias, ¿los objetos de qué clase se ejecutarán de forma concurrente? ¿Por tanto, qué agentes existen en esta simulación?

1.4. Lectura de archivos y lanzamiento de la simulación

Ahora fíjese en el archivo `er_data.txt`, que se encuentra en la carpeta `er`. Este archivo contiene la información necesaria para simular un servicio de Urgencias concreto. En él aparecen varias secciones:

```
AREA  80  80 1000 Admision
AREA 160 300 1000 Triaje
AREA 160 190 1000 SalaEspera
AREA 320  80 1000 Box_Trauma
...
```

Así como pacientes y sus protocolos:

```
...
PATIENT 1 Admision
TRANSFER 1 Triaje 3000
TRANSFER 1 SalaEspera 3000
TRANSFER 1 Box_Trauma 3000
TRANSFER 1 SalaEspera 3000
TRANSFER 1 Box_Trauma_Final 3000
...
```

Observe el código del método `readFile(Scanner in)` de la clase `EmergencyRoom`, que es el que procesa estos archivos.

Responda a las siguientes preguntas:

- ¿En las líneas que empiezan por la palabra "AREA", qué significan cada uno de los números y el texto que la siguen?
- ¿En las líneas que empiezan por la palabra "PATIENT", qué significan el número y el texto que la siguen?
- ¿En las líneas que empiezan por la palabra "TRANSFER", qué significan los dos números y el texto que la siguen?

1.5. Interfaz gráfica

Dentro del paquete `es.upm.dit.aled.lab4.er.gui` se encuentran las clases necesarias para ejecutar una interfaz gráfica que permite visualizar el movimiento de los pacientes por el servicio de Urgencias. Estas clases son:

- `Position2D`, que simplemente encapsula las coordenadas x e y, necesarias para representar un punto en un espacio bidimensional.
- `EmergencyRoomGUI`, que crea una ventana y dibuja dentro de ella cajas para representar las secciones y puntos para representar a los pacientes. Aunque no es necesario que entienda cómo está implementado este código, sí que es importante que se fije en la existencia de tres métodos, pues tendrá que utilizarlos durante la práctica:
  - `getInstance()`: Este método **estático** devuelve la referencia a un objeto de la clase `EmergencyRoomGUI`, que necesitará para ejecutar los dos métodos **no estáticos** que aparecen a continuación.
  - `animateTransfer(Patient p, Transfer t)`: Este método ejecuta la animación que mueve al `Patient` proporcionado desde su ubicación actual (`location`) hasta el destino del `Transfer` (`to`). **Tarda en ejecutarse tanto como dure el traslado.**
  - `removePatient(Patient p)`: Este método borra al `Patient` proporcionado de la pantalla, y debe ser invocado cuando un paciente ha terminado su protocolo.

Sirva de ejemplo del uso de estos métodos la siguiente línea de código, que ejecuta la animación de un paciente:

```
EmergencyRoomGUI.getInstance().animateTransfer(patient, transfer);
```

Patrón singleton

Quizá se esté preguntando por qué demonios tiene que usar el método `getInstance()` en lugar de un constructor para obtener la referencia a un objeto `EmergencyRoomGUI`. De hecho, ¡el constructor de `EmergencyRoomGUI` es privado! ¡Qué locura es esta!

Pues bien, esta forma de estructurar una clase (constructor privado y métodos `initialize()` y `getInstance()`) es típica de un patrón de diseño (una forma de estructurar programas) que recibe el nombre de *singleton*. Se usa, como su nombre sugiere, cuando queremos que solo sea posible crear una única instancia de una clase, que debe ser compartida por todos los que quieran usarla. En este caso, se ha hecho así porque siempre vamos a tener una única interfaz gráfica para la simulación. Ni más, ni menos.

1.6. Objetivo y descarga de la práctica

Este práctica, a diferencia de las anteriores, tiene dos objetivos:

- El primero, que ya conoce, es la simulación de un servicio de Urgencias de un hospital usando concurrencia. Esta parte de la práctica se desarrolla en la Sección 2.
- El segundo es la aceleración del algoritmo de búsqueda lineal de cadenas de nucleótidos en un genoma que desarrolló en la práctica 3. Esta aceleración será posible gracias a la ejecución en paralelo, usando tantas hebras como procesadores tenga su ordenador. Esta parte de la práctica se desarrolla en la Sección 3.

Sí, esta práctica es un poco más larga de lo normal. Pero, a cambio, la siguiente (la práctica 5) será un poco más corta. Se lo prometo.

La práctica está disponible en GitHub, en la siguiente dirección de Internet (URI):
https://github.com/rgarciacarmona/ALED-lab4
Para trabajar en ella, siga estos pasos:

1. Haga un *fork* de este repositorio a su cuenta de GitHub.
2. Antes de empezar a hacer *commits*, compruebe en *Settings* → *General* de su *fork* que la visibilidad es **pública**. Solo se evaluarán los *commits* hechos sobre un *fork* público.
3. Clone su *fork* e impórtelo en Eclipse. Para ello, seleccione *File* → *Import...* y, de entre las opciones disponibles, *Git* → *Projects from Git* → *Clone URI*. A continuación, introduzca la URI de su *fork* (`https://github.com/`*usuario*`/ALED-lab4`, donde *usuario* es su nombre de usuario de GitHub), no la del repositorio original. Después, haga click en *Next* tantas veces como sea necesario y, por último, en *Finish*.
4. Trabaje, haga *commit* de sus cambios y haga *push* a su *fork*.

Una vez descargado el código de la práctica, verá que este está compuesto únicamente de dos paquetes (uno de ellos con, a su vez, un subpaquete):

- `es.upm.dit.aled.lab4.er`: Que corresponde a la primera parte de la práctica.
- `es.upm.dit.aled.lab4.genome`: Que corresponde a la segunda parte de la práctica.

Enhorabuena, ya ha terminado con el trabajo previo a la sesión de laboratorio. Las secciones de la 2 en adelante puede realizarlas en clase.

2. Simulación de un servicio de Urgencias

En esta primera parte de la práctica, terminará de implementar el código de un simulador del servicio de Urgencias de un hospital. Como ya se habrá percatado al contestar las preguntas de la Sección 1, este es un sistema complejo que emerge de la interacción de varios agentes: los pacientes.

Antes de continuar, repase de nuevo el diagrama que aparece en la Figura 2, que contiene las clases del paquete `es.upm.dit.aled.lab4.er` que modelan el servicio de Urgencias. Es bueno que las tenga frescas en la cabeza.

2.1. Ejecución de la simulación

Para lanzar la simulación, deberá ejecutar el método `main()` de la clase `Simulation`. Este método crea una nueva instancia de `Simulation`, pasándole como parámetro el primer argumento que recibe el programa al ejecutarse. Inmediatamente después, llama al método `start()` del objeto de `Simulation` que acaba de crear.

Fíjese en el código del constructor de `Simulation`:

```
/**
 * Simulates an emergency room from a provided file
 */
public Simulation(String file) throws FileNotFoundException {
    // Initializes EmergencyRoom
    this.er = new EmergencyRoom();
    // Loads data from file
    File inputFile = new File(file);
    er.readFile(new Scanner(inputFile));
    // Initializes the GUI
    EmergencyRoomGUI.initialize(er);
}
```

Responda a la siguiente pregunta:

- ¿Cuál debe ser el primer argumento del programa si queremos que se ejecute el servicio de Urgencias que se describe en la Sección 1.4?

A continuación, cree una configuración de lanzamiento que tenga como resultado la ejecución del servicio descrito en la Sección 1.4 y ejecútela.

Debería ver algo muy similar a la pantalla que aparece en la Figura 3.

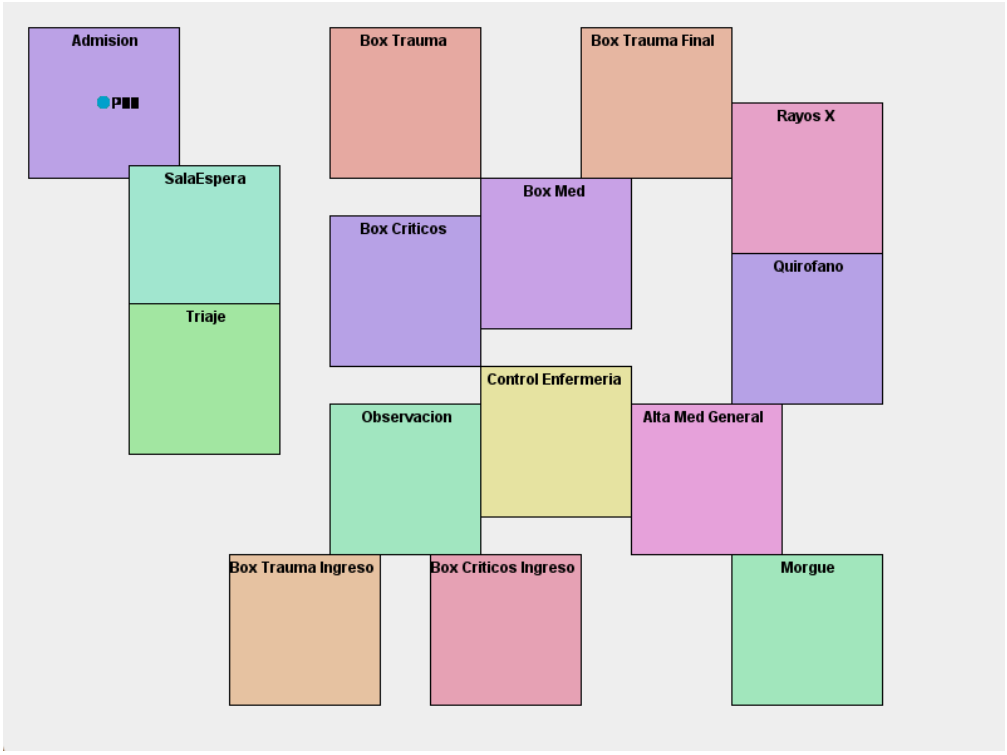


Figura 3: Simulación del servicio de Urgencias.

Como puede observar, más allá de mostrar las secciones del servicio y a todos los pacientes en Admisión, no ocurre nada. Esto se debe a que dos de las clases, `Patient` y `EmergencyRoom` aún tienen métodos sin implementar. Le toca a usted encargarse de ello.

2.2. Implementación de la clase `Patient`

`Patient` modela a los pacientes del hospital, que pueden realizar dos "acciones":

- **Ser atendidos en la ubicación en la que se encuentran:** Este comportamiento se produce en el interior del método `attendedAtLocation()`, y es muy sencillo: el paciente simplemente tiene que esperar el tiempo correspondiente al `Area` en la que se encuentre en ese momento.
- **Avanzar al siguiente paso en su protocolo:** Este comportamiento se produce en el interior del método `advanceProtocol()`. Se compone de los siguientes pasos:

  1. Pedir a la interfaz gráfica `EmergencyRoomGUI` que ejecute una animación que lleve el punto que representa al paciente hasta su nueva ubicación.
  2. Cambiar la ubicación actual del paciente.
  3. Avanzar el protocolo hasta el siguiente paso.

  Tenga en cuenta que este método, a diferencia de `attendedAtLocation()` **no debe esperar**; la espera ya la realiza `EmergencyRoomGUI` al ejecutar la animación.

Implemente estos dos métodos.

Trazas

Para facilitarle la tarea de arreglar los errores que, inevitablemente, tendrá su código, considere hacer que los pacientes impriman por pantalla su número y el comportamiento que están ejecutando en un momento dado. Créame, cuando tenga 20 pacientes interactuando a la vez lo va a agradecer.

`Patient` extiende la clase `Thread`, por lo que también deberá implementar el método `run()`. Este método es el que se ejecuta cuando alguien arranca las hebras de los pacientes y, por tanto, deberá utilizar los métodos que acaba de programar para representar el comportamiento de un paciente, que será el siguiente:

1. Ser atendido en la ubicación actual.
2. Avanzar al siguiente paso en su protocolo.
3. Vuelta al punto 1.

Este proceso termina cuando se ha llegado al último paso del protocolo. Tras ser atendido en esta última localización, el paciente deberá pedir a `EmergencyRoomGUI` que lo elimine, para que el punto que representa al paciente desaparezca de la pantalla.

2.3. Implementación de la clase `EmergencyRoom`

Ahora que ha terminado de implementar los pacientes, ha llegado el momento de que alguna parte del programa los maneje como los `Threads` que son. Y esa parte es `EmergencyRoom`. En esta clase hay dos métodos que deberá implementar:

- `admit(Patient patient)`, que comienza la ejecución del `Patient` que se le pasa como parámetro.
- `waitForDischarge(Patient patient)`, que se queda esperando a que el `Patient` que se le pasa como parámetro finalice.

Cuando termine, ejecute de nuevo la simulación. Si lo ha hecho todo correctamente, podré contemplar como dos decenas de pacientes campan a sus anchas por el servicio de Urgencias.

Enhorabuena, ha terminado la **primera parte** de la práctica. Tómese un breve descanso y a por la segunda parte.

3. Aceleración de un algoritmo de búsqueda en un genoma

La concurrencia no solo es útil para la simulación de sistemas complejos, sino también para aprovechar el paralelismo inherente a las arquitecturas *hardware* de los ordenadores actuales, pudiendo así resolver ciertos problemas computacionales en un tiempo menor.

Para ilustrar esto, en la **segunda parte** de esta práctica va a reimplementar el algoritmo que desarrolló en la Sección 2 de la práctica 3. Así que, antes de nada, abra dicha práctica y repase tanto el enunciado como el código que programó usted mismo (esperemos).

La idea es crear una nueva versión de su búsqueda lineal, de tal manera que se lancen tantas tareas (hebras que devuelven un resultado) como procesadores (o núcleos) posea el equipo en el que está trabajando, dividiéndose el espacio de búsqueda de forma equitativa entre ellas. Si, por ejemplo, su ordenador dispone de 8 procesadores, el programa resultante lanzará 8 tareas, encargándose cada una de las cuales de buscar en una octava parte del genoma. En el ejemplo, esta estrategia debería lograr que el tiempo de búsqueda se reduzca a aproximadamente una octava parte del original.

Fíjese en las clases del paquete `es.upm.dit.aled.lab4.genome`:

- `FASTAException`: una clase hija de `Exception` cuyo propósito es identificar que una excepción tiene su origen en el procesado de los datos obtenidos a partir de un archivo FASTA. Esta clase **no cambia respecto a la práctica 3**.
- `FASTASearchCallable`: una clase que ejecutará la búsqueda lineal (como en la práctica 3), pero que trabajará sobre un fragmento del genoma en lugar de sobre su totalidad. Esta clase contendrá el código equivalente a los métodos `search()` y `compare()` de la práctica 3. Además, esta clase implementa el interfaz `Callable`, ya que es una tarea que debe poder ser lanzada como hebra.
- `FASTAReaderThreads`: una clase que ejecutará tantas tareas `FASTASearchCallable` como procesadores tenga el equipo, indicando a cada una sobre qué parte del genoma debe buscar. Según estas vayan terminando, `FASTAReaderThreads` irá agregando los resultados en un único conjunto, que será el que devuelva. Esta clase reutiliza gran parte del código de la clase `FASTASearch` de la práctica 3 (todo menos los métodos `search()` y `compare()`.

3.1. Implementación de la clase `FASTASearchCallable`

La clase `FASTASearchCallable` lleva a cabo una búsqueda lineal dentro de un rango `[lo,hi)` del array de bytes del genoma (`content` en `FASTAReaderThreads`). Por tanto, el constructor de `FASTASearchCallable` tendrá el siguiente aspecto:

```
/**
 * Creates a new FASTASearchCallable that performs a linear search inside the
 * [lo, hi) range of the genome.
 *
 * @param reader  FASTAReaderThreads that contains the genome (content) and how
 *                many bytes (validBytes) of this content are valid. Usually
 *                the one that has called this constructor.
 * @param lo      The lower bound of the segment of content to be searched.
 * @param hi      The higher bound of the segment of content to be searched.
 * @param pattern The pattern to be found.
 */
public FASTASearchCallable(FASTAReaderThreads reader,
                    int lo, int hi, byte[] pattern) {
    // TODO
}
```

Implemente este constructor. Al hacerlo, sea consciente de que la razón de que se le pase una referencia a `FASTAReaderThreads` es porque el método `compare()` necesita acceder al contenido del genoma mediante el método `getContent()` de `FASTAReaderThreads`.

Copia de arrays

Al constructor de `FASTAReaderThreads` no se le pasa una copia de su sección del genoma, sino simplemente los límites superior e inferior. La razón tras esto es que hacer una copia de un array tan grande implica un gran impacto en el rendimiento. Además, no pasa nada porque varias tareas accedan al mismo array, ya que solo lo hacen para leer, y no para escribir.

Ahora fíjese en el método `call()`. Este método es el que ejecutarán de forma independiente las tareas `FASTASearchCallable`, y deberá devolver un `List` de `Integers` con las posiciones en las que se ha encontrado el patrón dentro de la sección del genoma que le corresponde a la tarea.

Este método **realiza la misma función** que el método `search()` de la clase `FASTAReader` que programó en la práctica 3. Por tanto, **la gran mayoría del código es idéntico**. La única diferencia realmente importante es que ahora no se busca en todo el genoma, sino solo en el segmento que se configuró en el constructor.

Implemente el método `call()`. Para ello, use el método `compare()` de esta misma clase, que ya ha sido implementado.

Compare mejorado

En realidad, el método `compare()` que se le proporciona no es el `compare()` original, sino la versión mejorada que deja de comparar en cuanto se encuentra un nucleótido que no coincide.

3.2. Implementación de la clase `FASTAReaderThreads`

Vamos, que ya falta poco. Para terminar la segunda parte de la práctica ya solo necesita implementar el método `search()` de la clase `FASTAReaderThreads`. Como se comentó anteriormente, este método ya no lleva a cabo la búsqueda por sí mismo, sino que lo que hace es crear tantas tareas `FASTASearchCallable` como procesadores tenga el ordenador, repartir el trabajo entre ellas y agregar los resultados.

Para implementarlo este método, tenga en cuenta los siguientes aspectos importantes:

- Para obtener el número de procesadores de su ordenador puede escribir el siguiente código:

```
int cores = Runtime.getRuntime().availableProcessors();
```

- Para crear un `ExecutorService` con un número fijo de hebras puede escribir el siguiente código:

```
ExecutorService executor = Executors.newFixedThreadPool(cores);
```

- El número de procesadores afecta a qué segmentos del genoma (`lo` y `hi`) le corresponde a cada tarea.
- Las tareas devuelven un `Future`, que no se concretará en una verdadera `List<Integer>` hasta que no se llame al método `get()` del `Future`.
- Al terminar de ejecutar un `ExecutorService` debe cerrarlo:

```
executor.shutdown();
```

3.3. Ejecución de la búsqueda lineal paralelizada

Por último, cree una configuración de lanzamiento que utilice la búsqueda lineal paralelizada que acaba de crear (método `main()` de la clase `FASTAReaderThreads`) sobre el archivo `chr19.fa` que se encuentra en la carpeta `cromosomes`, buscando la cadena de nucleótidos "GATTACA-GACA" (la misma que en la práctica 3). Apunte los tiempos que ha obtenido y compárelos con los que obtuvo en la práctica 3. Si no los apuntó en su momento, vuelva a ejecutarla.

Responda a las siguientes preguntas:

- ¿Cuántos procesadores tiene su equipo?
- ¿Cuántas hebras está usando su nueva búsqueda lineal?
- ¿Cuánto se ha reducido el tiempo de ejecución de su nueva búsqueda lineal en comparación con la original? ¿Era lo que esperaba?

Enhorabuena. Ahora, sí, ha terminado la práctica.

4. Extras

Para esta práctica no se proponen extras. Bastante ha tenido ya.